Objective: Students will explore how data structures model real-world engineering systems, learn to ingest and analyze simulation data, train a predictive model, and pitch their findings to an academic audience.

Week at a Glance 1
Day 1: WNTR/EPANET Recap & Problem Ideation 3
Part 1: Your UIC Project Recap (Instructor Demo) 3
Part 2: Student Brainstorming & Examples 3
Day 2: Data Acquisition & Ingestion 3
Focus: From GitHub to Google Colab 3
Tasks: 4
Day 3: Exploratory Data Analysis (EDA) 4
Focus: Understanding the Data Structure 4
Lab Activities: 4
Day 4: Statistical Machine Learning (The 80/20 Rule) 4
Focus: Predictive Modeling 4
Step-by-Step Implementation: 4
Day 5: Showcase & Elevator Pitches 5
Logistical Breakdown (60-Minute Class Window) 5

Day 1: WNTR/EPANET Recap & Problem Ideation

Part 1: Your UIC Project Recap (Instructor Demo)

Remind students how your project leverages Python, Google Colab, and **WNTR (Water Network Tool for Resilience)** to simulate water distribution networks based on **EPANET**.

The Core Concept: A water network is fundamentally a **Graph data structure**.

- **Nodes (Vertices):** Junctions, tanks, and reservoirs (storing attributes like elevation, demand, and pressure).
- **Links (Edges):** Pipes, pumps, and valves (storing attributes like length, diameter, and roughness).

Colab Workflow Recap:

1. **Environment Setup:** !pip install wntr
2. **Network Generation:** Loading an .inp file (EPANET format) into a WNTR network model object.
3. **Simulation:** Running a hydraulic simulation (wntr.sim.EpanetSimulator) over a specific time horizon.
4. **Data Extraction:** Pulling simulation results out of the network object into Pandas DataFrames (node pressures, link flow rates).

Part 2: Student Brainstorming & Examples

Students must identify an engineering or CS simulation problem they want to solve. To guide them, provide these starting research domains:

- **Smart Grid / Power Systems:** Simulating electrical grid loads, blackouts, or renewable energy integration (Nodes = Substations, Edges = Power lines).
- **Traffic & Urban Mobility:** Simulating traffic jams, traffic light optimization, or public transit routing (Nodes = Intersections, Edges = Roads).
- **Structural Engineering:** Stress and strain simulation on bridges or buildings under various loads (Finite Element Analysis data structures).
- **HVAC & Thermal Systems:** Simulating fluid dynamics or temperature distribution in energy-efficient smart buildings.

Tonight's Homework: Submit a 3-sentence problem statement and a link to a potential GitHub repository or open dataset.

Day 2: Data Acquisition & Ingestion

Focus: From GitHub to Google Colab

Students transition from an idea to a working environment. They must find an open-source simulation tool or dataset (similar to how you leveraged the WNTR GitHub resources) and load it into Colab.

Tasks:

1. **Resource Hunting:** Students clone repositories, download .csv, .json, or domain-specific files (like .inp or .mat).
2. **Data Ingestion:** Write the Python code to load this raw data into structured formats (pandas.DataFrame, numpy.ndarray, or networkx.Graph).
3. **Sanity Check:** Print data shapes, data types, and check for missing values (df.info(), df.head()).

Deliverable: A Google Colab notebook shared with the instructor where the dataset successfully loads without errors.

Day 3: Exploratory Data Analysis (EDA)

Focus: Understanding the Data Structure

Before building a model, students must understand the underlying patterns, anomalies, and relationships within their simulation data.

Lab Activities:

- **Feature Engineering:** Identifying the *Independent Variables* (features like pipe diameter, time of day, initial load) and *Dependent Variables* (targets like pressure drop, traffic speed, failure probability).
- **Statistical Summary:** Calculating means, medians, standard deviations, and correlations.
- **Visualization:** Creating at least 3 plots (e.g., correlation heatmaps, time-series line graphs of the simulation, or network graph visualizations).

Day 4: Statistical Machine Learning (The 80/20 Rule)

Focus: Predictive Modeling

Students will train a baseline statistical or machine learning model to predict a simulation outcome based on their Day 3 analysis.

Step-by-Step Implementation:

1. **The 80/20 Split:** Explain the importance of separating data to prevent overfitting. Use scikit-learn to split the data.

2. Python
   from sklearn.model_selection import train_test_split
   X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

3. **Model Selection:** Keep it simple and accessible. Recommend standard models depending on their goal:
   - *Regression (Predicting a continuous number like pressure or speed):* Linear Regression or Random Forest Regressor.
   - *Classification (Predicting a state like fail/pass or traffic/clear):* Logistic Regression or Decision Trees.

4. **Evaluation:** Calculate performance metrics (e.g., Mean Squared Error for regression, or Accuracy/F1-Score for classification) on the 20% test set.

Day 5: Showcase & Elevator Pitches

Logistical Breakdown (60-Minute Class Window)

To keep 25 presentations moving efficiently, a strict schedule is paramount. Allowing **partners** reduces the total pitches to around 12–15, which offers a much more comfortable buffer.

- **0:00 – 0:05 (5 mins):** Instructor Intro & Welcoming the UIC Guest Professor.
- **0:05 – 0:10 (5 mins):** UIC Professor Opening Remarks (Inspiring students on the value of simulation research).
- **0:10 – 0:45 (35 mins): The 60-Second Elevator Pitches.**
    - *Format:* 1 slide per project. 45 seconds to speak, 15 seconds to transition.
    - *Pitch Structure:* Problem statement -> Data source/structure -> ML model results.
- **0:45 – 0:55 (10 mins):** UIC Professor Feedback & Closing Thoughts.
- **0:55 – 1:00 (5 mins):** Instructor Wrap-up and Next Steps.